

Transformaciones geométricas en imágenes

Visión Artificial (GIEC), Sistemas de Visión Artificial (GIC)

Transformaciones geométricas *Generalidades*

- ❑ Las **transformaciones geométricas** se utilizan para resaltar/optimizar ciertas **zonas** o regiones dentro de una imagen (**regiones de interés / regions of interest, ROIs**), modificar la geometría/coordenadas de una imagen para registro, rectificación, corrección de distorsión, cambio de escala, orientación, alineamiento, etc.
- ❑ Para ello se realizan operaciones que modifican las **coordenadas espaciales** de la imagen: operaciones geométricas.
- ❑ Algunas de estas transformaciones son: **traslaciones, rotaciones/giros, homotecias/zoom, shear, homografías, etc.**

Imagen 1. Original



Imagen 2.
Recorte



Imagen 3.
Giro



Imagen 4.
Recorte



Imagen 5. Zoom: Final



<http://www2.diariomotor.com/imagenes/2012/10/posts/impuesto-de-matriculacion-01.jpg> [2021]

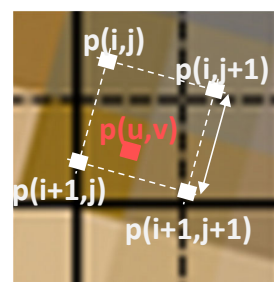
- ❑ El objetivo de una **operación geométrica** es **transformar** los valores de una imagen para que se vean como podrían observarse desde otro punto de vista.
- ❑ Las imágenes son **discretas**: los píxeles forman una rejilla, donde las coordenadas de cada celda son números **enteros** → entre dos píxeles no existen valores de intensidad.
- ❑ Cuando se aplica la transformación (un desplazamiento, giro o zoom), los píxeles originales quedarían **desplazados** a una nueva posición, por ejemplo, para el caso de la traslación:



Ejemplo de:
<http://slideplayer.es/slide/4333895/>

- ❑ Esta nueva posición puede tener **coordenadas reales**, que requieren **interpolación**.

- ❑ El proceso para llevar a cabo la transformación geométrica de una imagen se realiza en **2 pasos**:
 1. En 1^{er} lugar, es necesario **establecer la correspondencia entre coordenadas de entrada y salida**. Dependerá de la transformación: giro, traslación, homotecia... Cuando se parte de los píxeles de entrada para calcular su posición en la imagen de salida, se denomina **mapeo directo (forward mapping)**. Cuando para cada píxel de la imagen de salida, se calcula mediante la transformación inversa la posición de la imagen de entrada de la que procede, se denomina **mapeo inverso (inverse warping)**.
 2. Una vez conocemos **la correspondencia de los píxeles**, debemos calcular el **valor (nivel de gris/color) definitivo** que tendrán los píxeles de la imagen de salida: debemos calcular de qué color debe ser $p(u,v)$ en función de los colores de los píxeles cercanos en la imagen original → esa operación se denomina **interpolación**.

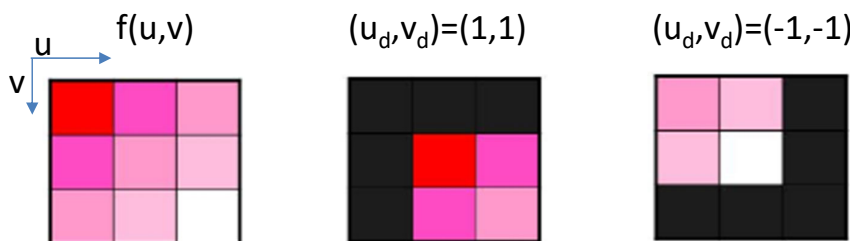


- Para **calcular las coordenadas de la imagen de salida** en las que acabarán los píxeles de la imagen de entrada en caso de perspectivas, rotaciones y traslaciones, es muy conveniente poder representar las transformaciones y las coordenadas de los píxeles de forma **matricial**.
 - Concretamente, si los puntos se representan en **coordenadas homogéneas**, las transformaciones geométricas se podrán realizar como **producto de matrices**. De hecho, utilizar estas coordenadas permite **componer una sucesión de transformaciones en una única matriz (por la propiedad asociativa del producto de matrices)**.
 - Dado un **punto en coordenadas cartesianas (x, y)** , para representarlo en **coordenadas homogéneas** se le añade una componente:

$$(x, y) \rightarrow (x', y', W)$$
 - El valor de W inicialmente es **1** y puede cambiar de valor al operar.
 - Dado un **punto en coordenadas homogéneas, (x', y', W)** , para **representarlo en coordenadas cartesianas** se realiza la siguiente operación:

$$(x, y) = (x'/W, y'/W)$$
 - Ej: El píxel (2,3) en coordenadas cartesianas se puede representar en coordenadas homogéneas como: (2, 3, 1), (4, 6, 2), (20, 30, 10)... Todas ellas serán representaciones **equivalentes** del mismo punto.

- La **traslación**: dado un píxel $f(u, v)$, este se desplaza (u_d, v_d) manteniéndose la intensidad $\rightarrow g(u_f, v_f) = f(u_i, v_i) = g(u_i + u_d, v_i + v_d) \rightarrow g(u_f, v_f) = f(u_f - u_d, v_f - v_d)$, donde (u_i, v_i) son las **coordenadas iniciales**, (u_f, v_f) las **finales**, y (u_d, v_d) el **vector desplazamiento**.



Ejemplo adaptado de:
<http://slideplayer.es/slide/4333895/>

- En coordenadas homogéneas, la posición de los píxeles tras la traslación se puede calcular realizando la siguiente operación matricial:

$$\begin{bmatrix} u_f \\ v_f \\ w \end{bmatrix} = \begin{bmatrix} 1 & 0 & u_d \\ 0 & 1 & v_d \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} u_i \\ v_i \\ 1 \end{bmatrix} \longrightarrow \begin{aligned} u_f &= u_i + u_d \\ v_f &= v_i + v_d \end{aligned}$$

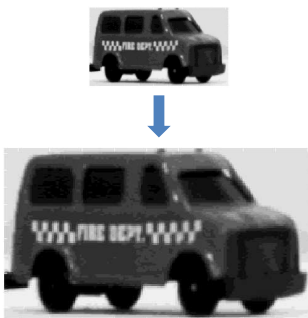
- ❑ **Homotecia/zoom:** Una homotecia escala (expande o reduce) una imagen respecto de un origen/centro. En ocasiones se combina con un *crop* que consiste en seleccionar una parte de la imagen (región de interés) y separarla del resto de la imagen original, para, posteriormente realizar la homotecia.

- ❑ Posibles matrices para la **expansión/reducción**:

$$\begin{bmatrix} u_f \\ v_f \\ w \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & a \end{bmatrix} \begin{bmatrix} u_i \\ v_i \\ 1 \end{bmatrix}$$

$$\begin{bmatrix} u_f \\ v_f \\ 1 \end{bmatrix} = \begin{bmatrix} b & 0 & 0 \\ 0 & b & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} u_i \\ v_i \\ 1 \end{bmatrix}$$

$$\begin{bmatrix} u_f \\ v_f \\ 1 \end{bmatrix} = \begin{bmatrix} c & 0 & 0 \\ 0 & d & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} u_i \\ v_i \\ 1 \end{bmatrix}$$



- ❑ Para multiplicar por 10 el tamaño de la imagen hay que mover los píxeles: $(1,1) \rightarrow (10,10)$, $(1,2) \rightarrow (10,20)$
- ❑ Para ello se pueden utilizar las siguientes matrices:

$$\begin{bmatrix} u_f \\ v_f \\ 1 \end{bmatrix} = \begin{bmatrix} 10 & 0 & 0 \\ 0 & 10 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} u_i \\ v_i \\ 1 \end{bmatrix}$$

Posición final: $(10 u_i, 10 v_i, 1)$

$\rightarrow (10 u_i, 10 v_i)$

$u_f = 10 u_i$

$v_f = 10 v_i$

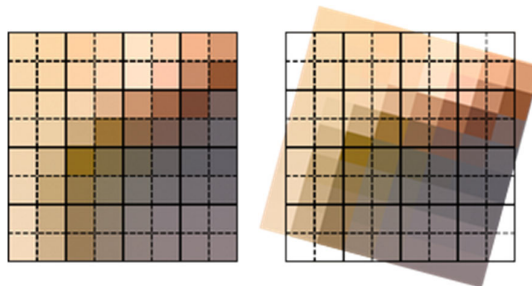
$$\begin{bmatrix} u_f \\ v_f \\ w \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0.1 \end{bmatrix} \begin{bmatrix} u_i \\ v_i \\ 1 \end{bmatrix}$$

Posición final: $(u_i, v_i, 0.1)$

$\rightarrow (u_i / 0.1, v_i / 0.1)$

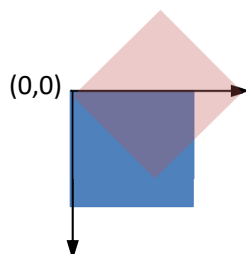
$\rightarrow (10 u_i, 10 v_i)$

- ❑ Los **giros** se utilizan para producir efectos estéticos, y para simular/corregir la **rotación** de la cámara o del propio objeto.



Ejemplo de: <http://www.kolor.com/wiki-en/Interpolation and blenders> [2021]

- ❑ Los parámetros necesarios para realizar la rotación son el **ángulo de giro** (θ) y las coordenadas del **centro de rotación**.
- ❑ La **rotación** cuando el **centro está en el origen de coordenadas (0,0)** viene dada por:



$$\begin{bmatrix} u_f \\ v_f \\ 1 \end{bmatrix} = \begin{bmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} u_i \\ v_i \\ 1 \end{bmatrix}$$

□ La **rotación de un ángulo (θ) respecto a un centro genérico** (u_d, v_d), que **no** esté en el origen (esquina superior izquierda), implica 3 pasos:

1. Traslación para que el centro de rotación esté en el origen ($-u_d, -v_d$).
2. Giro con ángulo θ .
3. Traslación para volver a ubicar el centro de rotación en la posición original (u_d, v_d).

$$\begin{bmatrix} u_f \\ v_f \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & u_d \\ 0 & 1 & v_d \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & -u_d \\ 0 & 1 & -v_d \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} u_i \\ v_i \\ 1 \end{bmatrix}$$

La multiplicación de matrices cumple la propiedad asociativa, por lo tanto, este producto se puede agrupar en una única matriz.

□ Transformación afín

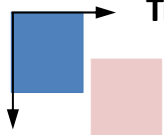
- En coordenadas homogéneas, es una transformación lineal donde M es una matriz 2×3 **arbitraria (obviamos la fila inferior, que sería $(0 \ 0 \ 1)$)**.
- Al aplicarse, **preserva el paralelismo** de las rectas, pero **no los ángulos**.

$$P' = M_A * P$$

$$\begin{pmatrix} u' \\ v' \end{pmatrix} = \begin{pmatrix} M_{11} & M_{12} & M_{13} \\ M_{21} & M_{22} & M_{23} \end{pmatrix} \begin{pmatrix} u \\ v \\ 1 \end{pmatrix}$$

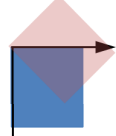
□ Las transformaciones que conocemos ya y algunas otras son **casos particulares** de la transformación afín:

Traslación



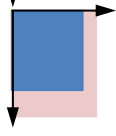
$$M_A = \begin{pmatrix} 1 & 0 & u_d \\ 0 & 1 & v_d \end{pmatrix}$$

Rotación



$$M_A = \begin{pmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \end{pmatrix}$$

Escalado (scale)



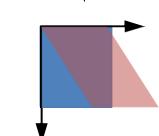
$$M_A = \begin{pmatrix} a & 0 & 0 \\ 0 & b & 0 \end{pmatrix}$$

Espejo (mirror)



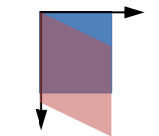
$$M_A = \begin{pmatrix} -1 & 0 & 0 \\ 0 & -1 & 0 \end{pmatrix}$$

Shear x (sesgo/cizallamiento en u)



$$M_A = \begin{pmatrix} 1 & A & 0 \\ 0 & 1 & 0 \end{pmatrix}$$

Shear y (sesgo/cizallamiento en v)



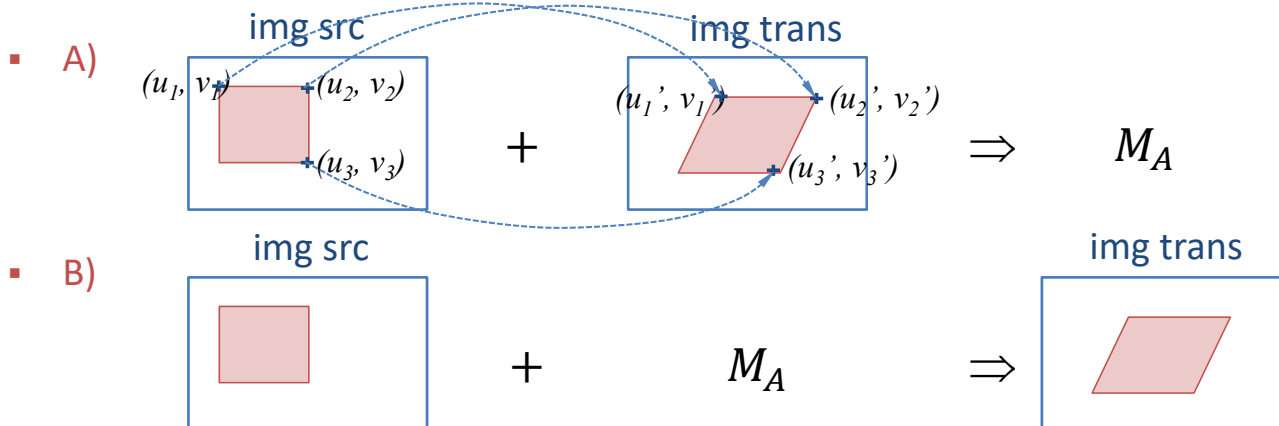
$$M_A = \begin{pmatrix} 1 & 0 & 0 \\ B & 1 & 0 \end{pmatrix}$$

$$P' = M_A * P$$

$$\begin{pmatrix} u' \\ v' \end{pmatrix} = \begin{pmatrix} M_{11} & M_{12} & M_{13} \\ M_{21} & M_{22} & M_{23} \end{pmatrix} \begin{pmatrix} u \\ v \\ 1 \end{pmatrix}$$

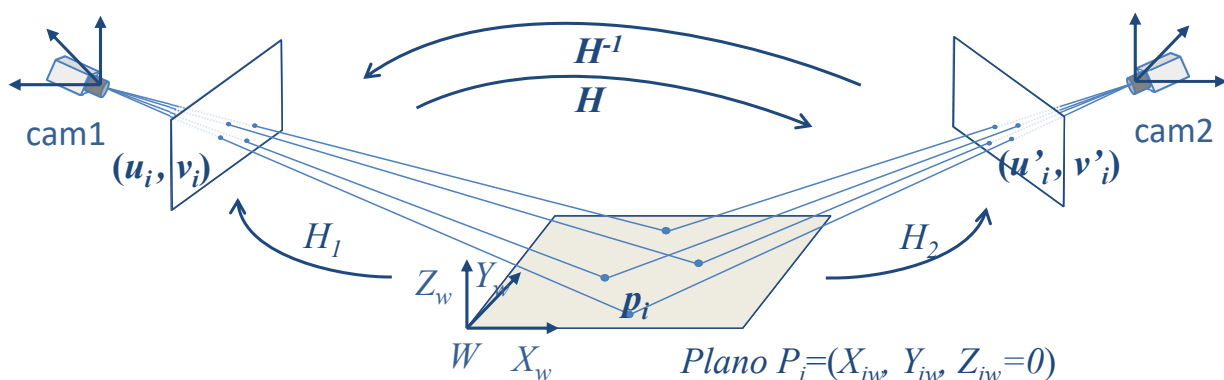
Transformación afín

- 6 variables: 6 grados de libertad.
- Si tenemos 2 imágenes y de una puede obtenerse la otra aplicando una transformación afín, podemos obtener la matriz M_A de transformación conociendo la posición de **tres pares de puntos que no pertenezcan a la misma línea** en ambas imágenes $\rightarrow$ nos darán 6 ecuaciones (3 de las componentes 'u' y 3 de las 'v') que nos permitirán despejar las 6 variables.



Transformación proyectiva

- Una **transformación proyectiva** relaciona geoméricamente dos **planos** mediante una proyección perspectiva.
- Una **homografía** es la transformación proyectiva invertible que permite relacionar los puntos de dos planos. En visión artificial se utiliza, por ejemplo, para relacionar las imágenes de un mismo plano de la escena vistas desde diferentes cámaras. En 2D, los términos transformación proyectiva y homografía suelen utilizarse como equivalentes.
- Es más general que una transformación afín.
- Preserva las rectas**: los puntos de una recta siguen estando sobre una recta.
- No preserva**, en general, distancias, ángulos ni el paralelismo (Ej: rectas que pueden aparecer en la foto de una fachada vista por 2 cámaras).



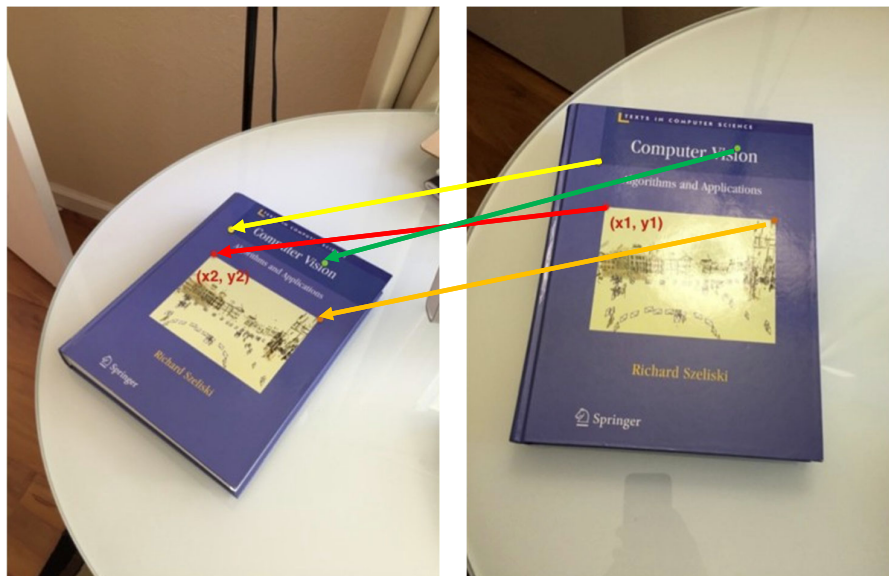
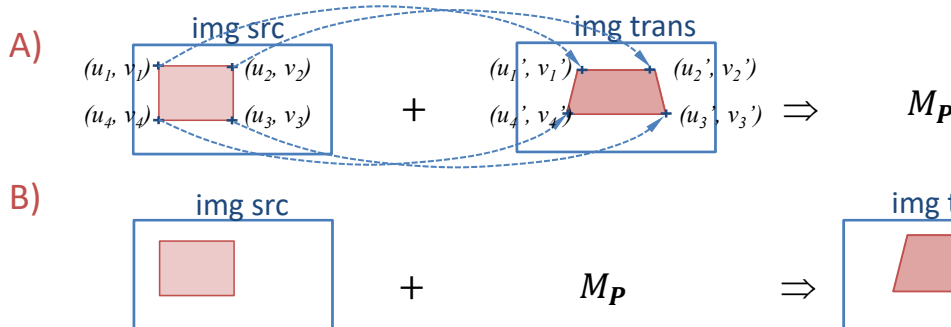
Transformación proyectiva

- La transformación de proyección se realiza multiplicando por una **matriz de proyección, M_p , 3×3** . Son 9 posiciones, pero M_{33} es un factor de escala, que se puede normalizar a $M_{33}=1$, por lo que la matriz tiene 8 grados de libertad (variables) y se puede obtener a partir de **cuatro pares de puntos** correspondientes a 4 puntos que en el mundo real estén sobre el mismo plano y no haya 3 colineales (configuración no degenerada).

$$P' = M_p * P$$

$$\begin{pmatrix} u' \\ v' \\ w \end{pmatrix} = \begin{pmatrix} M_{11} & M_{12} & M_{13} \\ M_{21} & M_{22} & M_{23} \\ M_{31} & M_{32} & M_{33} \end{pmatrix} \begin{pmatrix} u \\ v \\ 1 \end{pmatrix} \quad \begin{cases} u' = \frac{M_{11}u + M_{12}v + M_{13}}{M_{31}u + M_{32}v + M_{33}} \\ v' = \frac{M_{21}u + M_{22}v + M_{23}}{M_{31}u + M_{32}v + M_{33}} \end{cases}$$

H, H^{-1}, H_1 y H_2 son matrices que permiten realizar la homografía entre los distintos planos (escena e imágenes 1 y 2).

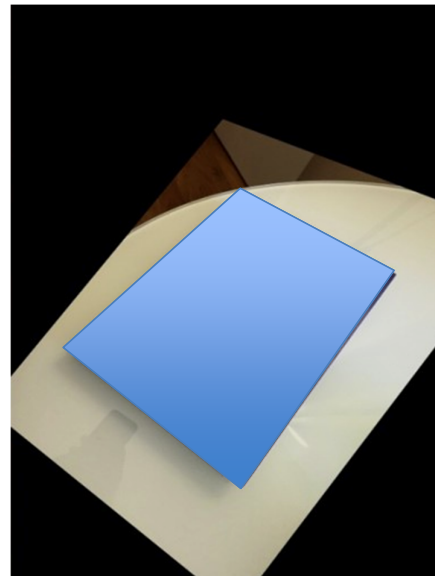


$$\begin{bmatrix} x_1 \\ y_1 \\ 1 \end{bmatrix} = H \begin{bmatrix} x_2 \\ y_2 \\ 1 \end{bmatrix} = \begin{bmatrix} h_{00} & h_{01} & h_{02} \\ h_{10} & h_{11} & h_{12} \\ h_{20} & h_{21} & h_{22} \end{bmatrix} \begin{bmatrix} x_2 \\ y_2 \\ 1 \end{bmatrix}$$



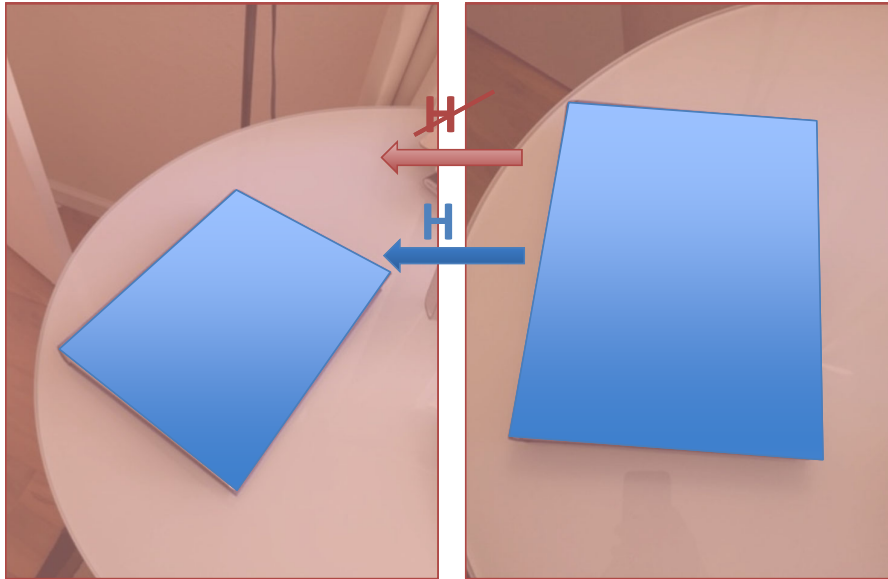
$$\begin{bmatrix} x_1 \\ y_1 \\ 1 \end{bmatrix} = H \begin{bmatrix} x_2 \\ y_2 \\ 1 \end{bmatrix} = \begin{bmatrix} h_{00} & h_{01} & h_{02} \\ h_{10} & h_{11} & h_{12} \\ h_{20} & h_{21} & h_{22} \end{bmatrix} \begin{bmatrix} x_2 \\ y_2 \\ 1 \end{bmatrix}$$

<https://www.learnopencv.com/homography-examples-using-opencv-python-c/>



$$\begin{bmatrix} x_1 \\ y_1 \\ 1 \end{bmatrix} = H \begin{bmatrix} x_2 \\ y_2 \\ 1 \end{bmatrix} = \begin{bmatrix} h_{00} & h_{01} & h_{02} \\ h_{10} & h_{11} & h_{12} \\ h_{20} & h_{21} & h_{22} \end{bmatrix} \begin{bmatrix} x_2 \\ y_2 \\ 1 \end{bmatrix}$$

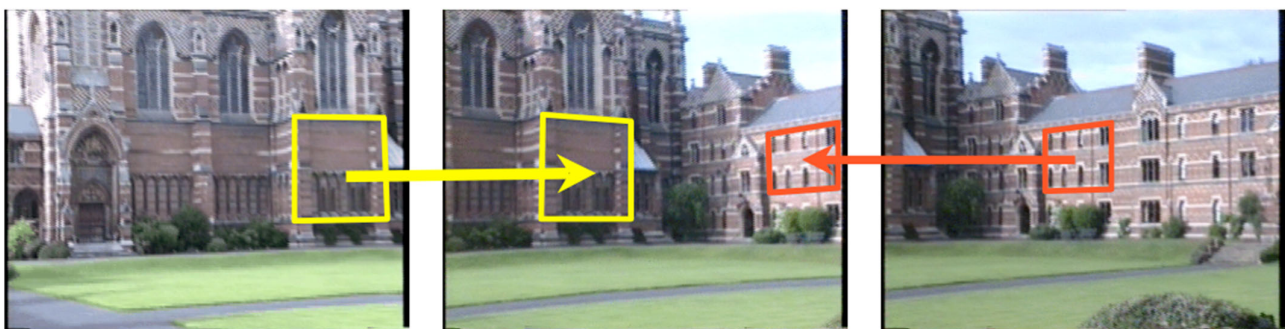
<https://www.learnopencv.com/homography-examples-using-opencv-python-c/>



$$\begin{bmatrix} x_1 \\ y_1 \\ 1 \end{bmatrix} = H \begin{bmatrix} x_2 \\ y_2 \\ 1 \end{bmatrix} = \begin{bmatrix} h_{00} & h_{01} & h_{02} \\ h_{10} & h_{11} & h_{12} \\ h_{20} & h_{21} & h_{22} \end{bmatrix} \begin{bmatrix} x_2 \\ y_2 \\ 1 \end{bmatrix}$$

<https://www.learnopencv.com/homography-examples-using-opencv-python-c/>

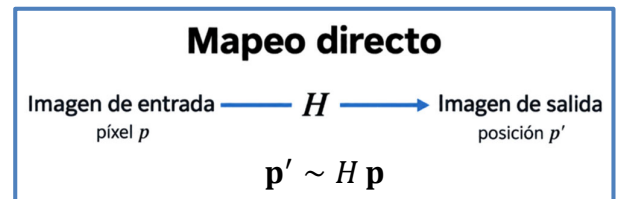
- Aplicación de homografías para crear imágenes panorámicas cuando la cámara rota sin traslación significativa



from Hartley & Zisserman

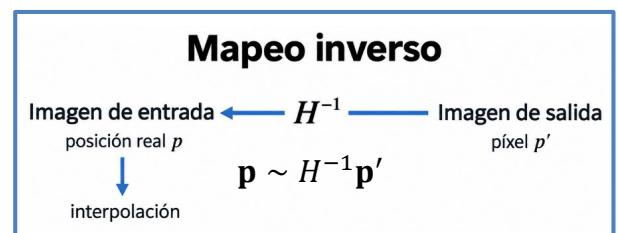
❑ Mapeo directo (*forward mapping*)

- ❑ Para cada píxel de la **imagen de entrada**, se calcula mediante la **transformación geométrica** la posición que le corresponde en la **imagen de salida**.
- ❑ Puede ocurrir que varios píxeles lleguen a posiciones próximas y que queden **huecos** en la imagen de salida.



❑ Mapeo inverso (*inverse warping*)

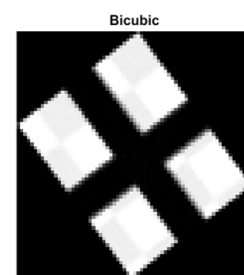
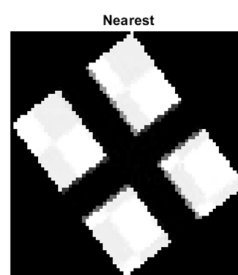
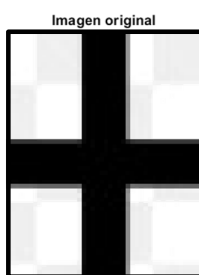
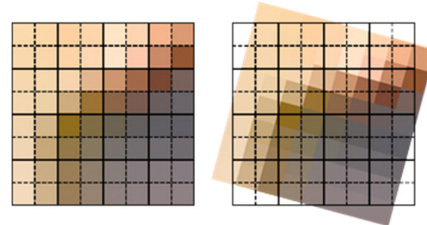
- ❑ Para cada píxel de la **imagen de salida**, se calcula mediante la **transformación inversa** la posición de la **imagen de entrada** de la que procede:
- ❑ La posición calculada normalmente no coincide exactamente con un píxel de entrada, por lo que su intensidad se obtiene mediante **interpolación**.
- ❑ **Ventaja:** evita los huecos que puede producir el mapeo directo.



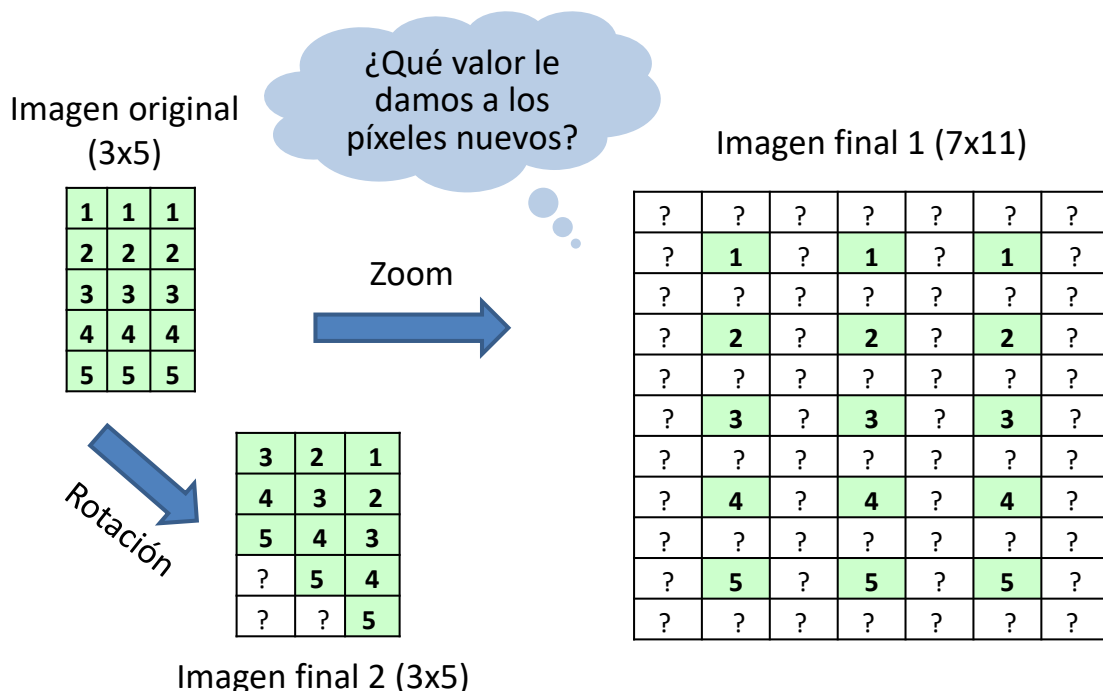
Transformation	Matrix	# DoF	Preserves	Icon
translation	$\begin{bmatrix} I & t \end{bmatrix}_{2 \times 3}$	2	orientation	
rigid (Euclidean)	$\begin{bmatrix} R & t \end{bmatrix}_{2 \times 3}$	3	lengths	
similarity	$\begin{bmatrix} sR & t \end{bmatrix}_{2 \times 3}$	4	angles	
affine	$\begin{bmatrix} A \end{bmatrix}_{2 \times 3}$	6	parallelism	
projective	$\begin{bmatrix} \tilde{H} \end{bmatrix}_{3 \times 3}$	8	straight lines	

- Una vez definida la **transformación geométrica**, debemos determinar los valores de intensidad de los píxeles de la imagen de salida. En general, las posiciones calculadas **no coinciden exactamente con las coordenadas de un píxel único** en la imagen de entrada, por lo que es necesario estimar su valor mediante **interpolación**.

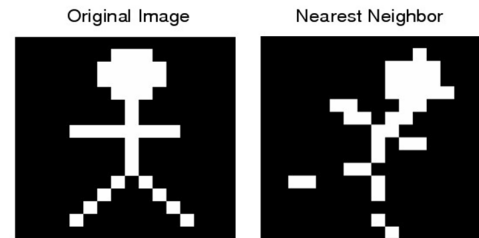
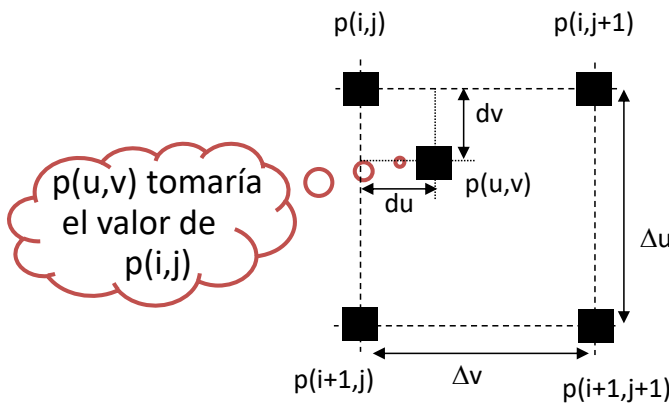
Ejemplo de: http://www.kolor.com/wiki-en/Interpolation_and_blenders [2021]



- Los **algoritmos de interpolación** determinan el nivel de intensidad de los píxeles de la imagen final a partir de uno o varios que lo rodean. **Se usa en giros, homotecias, etc.**



- El tipo más fácil de **interpolación** es la **interpolación al vecino más cercano (*nearest*)**: el píxel $p(u,v)$ de la imagen final toma el **valor del píxel más cercano** (distancia euclídea) de los cuatro vecinos que le rodean.



Ejemplo de: http://northstar-www.dartmouth.edu/doc/idl/html_6.2/Interpolation_Methods.html

- Una forma de **expansión** frecuente, por ejemplo, para agrandar una imagen (zoom): **interpolación lineal** (valor medio entre dos píxeles).
- Ejemplo de ampliación x 2**: permite pasar de imágenes de $N \times N$ a imágenes de $(2N-1) \times (2N-1)$ y se puede repetir las veces que se quiera.

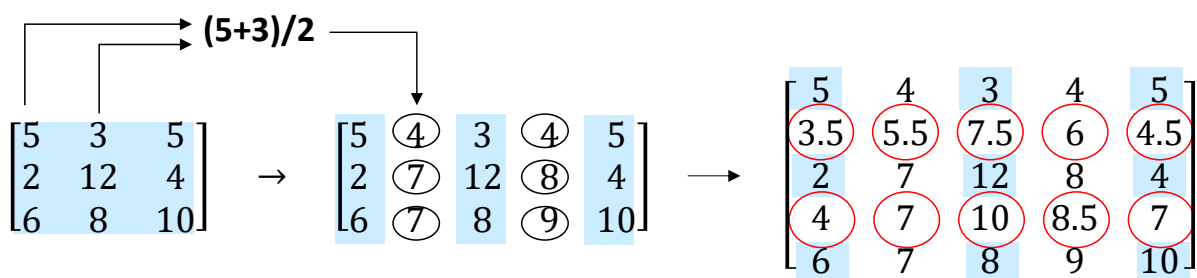


Imagen original

Imagen con filas expandidas

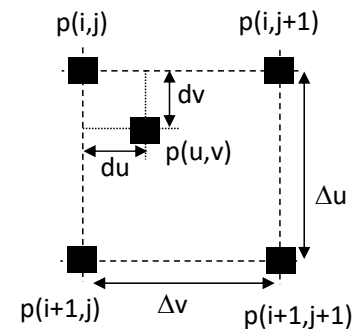
Imagen con filas y columnas expandidas

- La **interpolación bilineal** asigna a cada píxel el **valor medio ponderado** de las intensidades de los **cuatro** píxeles que le rodean.
- Los **factores de ponderación** vienen dados por la distancia entre el píxel y los del entorno:

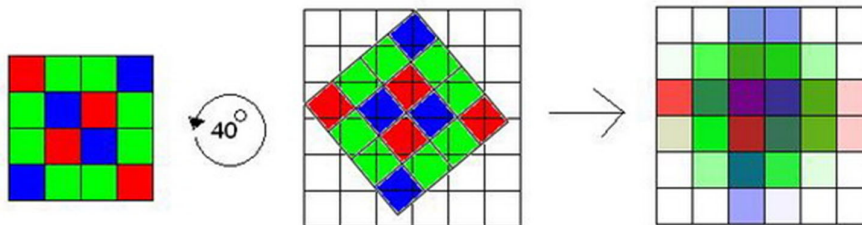
$$p(u, v) = a_1 p(i, j) + a_2 p(i, j + 1) + a_3 p(i + 1, j) + a_4 p(i + 1, j + 1)$$

$$a_1 = (1 - du)(1 - dv); \quad a_2 = du(1 - dv)$$

$$a_3 = (1 - du)dv; \quad a_4 = dudv$$



- Produce normalmente transiciones más suaves que vecino más cercano, a costa de cierto suavizado y mayor cálculo.



Ejemplo de:
<http://www.codeproject.com/Articles/12230/Anti-Aliased-Image-Rotation>

- En la **interpolación bicúbica**, el valor del píxel interpolado se calcula combinando los valores de los **16 píxeles más cercanos** mediante polinomios cúbicos.
- Suele producir resultados **más suaves** que la bilineal.
- Tiene **mayor coste computacional** que la interpolación bilineal.

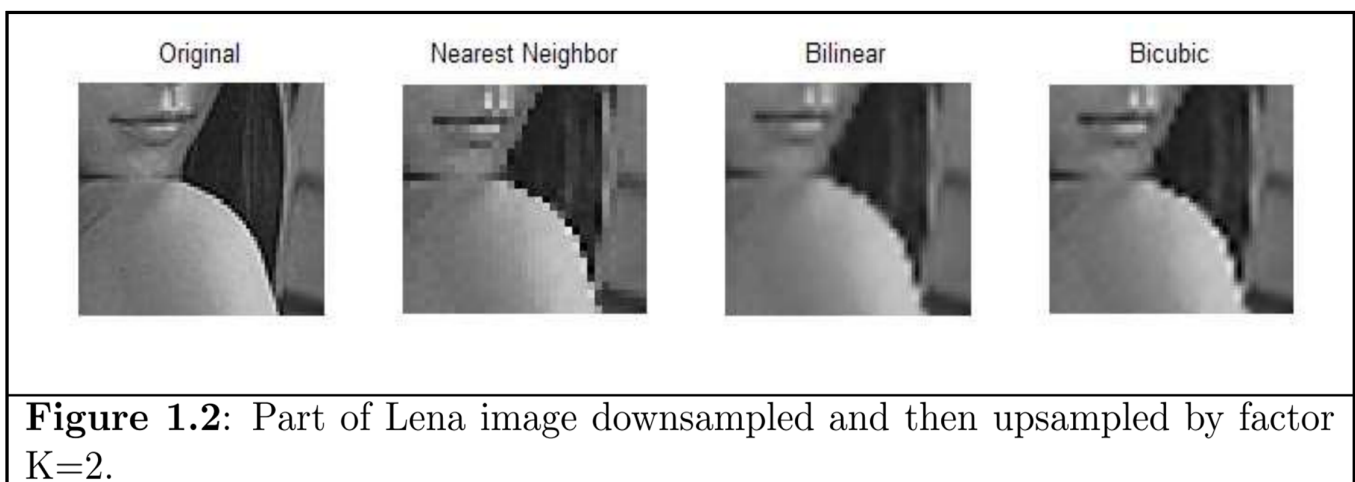


Figure 1.2: Part of Lena image downsampled and then upsampled by factor $K=2$.

Operaciones Morfológicas

Visión Artificial (GIEC), Sistemas de Visión Artificial (GIC)

Índice

- ☐ Operaciones morfológicas:
 - ☐ Dilatación
 - ☐ Erosión
 - ☐ Gradiente morfológico
 - ☐ Cierre (*Closing*)
 - ☐ Apertura (*Opening*)
 - ☐ *Top-hat (White top-hat)*
 - ☐ *Bottom-hat (Black top-hat)*

- ❑ Las **operaciones morfológicas** implican desplazar un elemento estructurante o *kernel* (B) por toda la imagen (A) y realizar las operaciones específicas sobre la superposición entre A y B, y asignar el resultado al píxel central (**punto de anclaje**).
- ❑ Para el caso binario, el **elemento estructurante** define el entorno de vecindad a través de una matriz de "1s" y "0s".
 - ❑ El píxel central (**punto de anclaje**) representa el píxel de interés.
 - ❑ Los elementos de la matriz que están en "1" definen el **entorno de vecindad**.
- ❑ Ejemplos de elementos estructurantes:

1	1	1
1	1	1
1	1	1

0	1	0
1	1	1
0	1	0

1	0	1
0	1	0
1	0	1

0	1	0
0	1	0
0	1	0

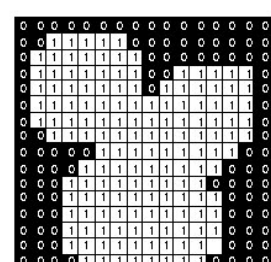
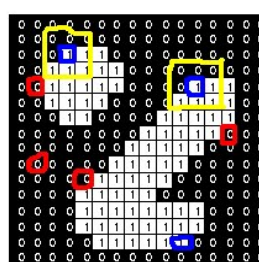
0	0	0
1	1	1
0	0	0

❑ Dilatación

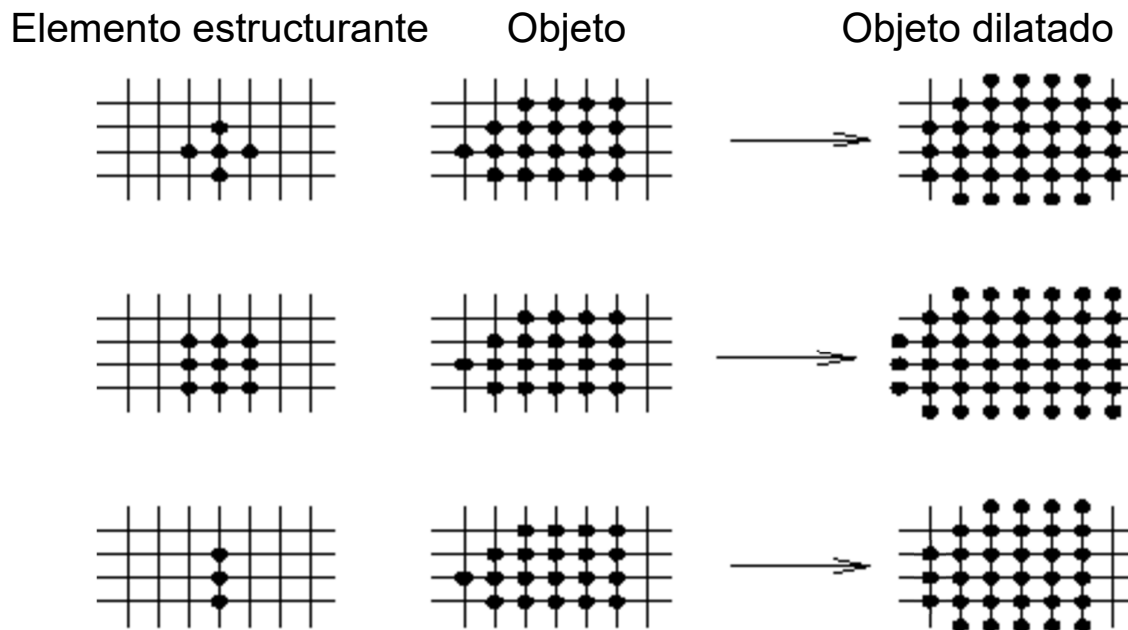
- ❑ **Regla:** Dada una imagen A, el píxel central del elemento estructurante B tomará, en la imagen de salida, el **valor mayor** de A cubierto por B. Si la imagen original es en blanco y negro, la imagen de salida es:

$$A \oplus B = \{x | [B_x \cap A] \neq \emptyset\}$$

- ❑ Si el primer plano es brillante sobre un fondo oscuro, entonces:
 - ❑ El **efecto** de la dilatación es "**engordar**" los objetos.
 - ❑ **Reduce o puede cerrar pequeños huecos y discontinuidades dependiendo del elemento estructurante.**
 - ❑ Si se aplica varias veces, el fondo desaparece.



- Si un objeto se dilata con **diferentes** elementos estructurantes, el resultado es distinto.



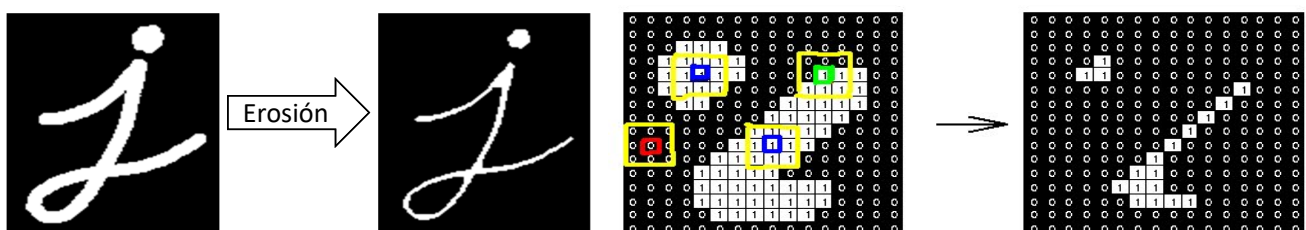
http://utam.gg.utah.edu/tomo03/03_mid/HTML/node103.html

Erosión

- Regla:** Dada una imagen A, el píxel central del elemento estructurante B tomará, en la imagen de salida, el **valor menor** de A cubierto por B. Si la imagen original es en blanco y negro, la imagen de salida es:

$$A \ominus B = \{x \mid B_x \subseteq A\}$$

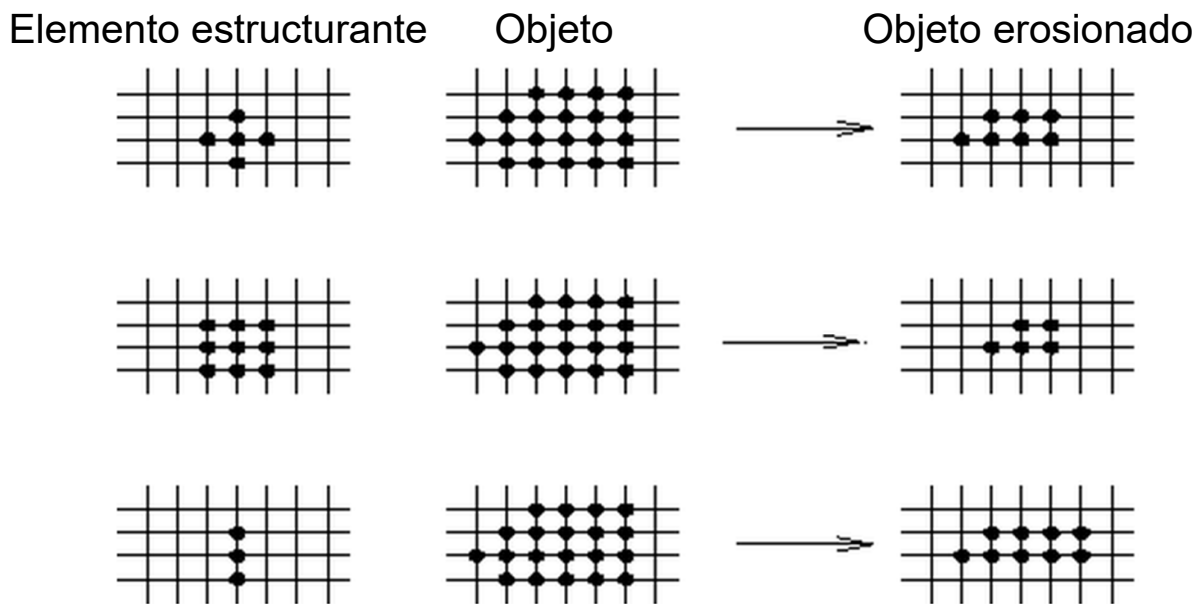
- Si el primer plano es brillante sobre un fondo oscuro, entonces:
 - El **efecto** de la erosión es "**adelgazar**" los objetos.
 - Elimina estructuras** en las que el elemento estructurante no puede quedar contenido.
 - Si se aplica varias veces, todos los objetos desaparecen.



https://docs.opencv.org/2.4/doc/tutorials/imgproc/erosion_dilatation/erosion_dilatation.html

<http://chilmania.blogspot.com.es/2010/06/matlab-code-for-dilation-and-erosion.html>

- Si un objeto se erosiona con **diferentes** elementos estructurantes, el resultado es distinto.



http://utam.gg.utah.edu/tomo03/03_mid/HTML/node103.html

Gradiente morfológico

- Es la **diferencia** entre la **dilatación** y la **erosión** de una imagen.

$$I = (A \oplus B) - (A \ominus B)$$

- Es útil para encontrar el **contorno** de un objeto, como se puede ver en la imagen



https://docs.opencv.org/3.2.0/d9/d61/tutorial_py_morphological_ops.html

❑ Closing (Cierre)

- ❑ Es una dilatación seguida de una erosión.

$$A \bullet B = (A \oplus B) \ominus B$$

- ❑ **Rellena/cierra** los pequeños **agujeros** que existen en la imagen, si el primer plano es brillante sobre un fondo oscuro.
- ❑ Satisface la propiedad de la **idempotencia**: después de la primera aplicación, otras aplicaciones no producen ningún cambio en el resultado.



https://docs.opencv.org/3.2.0/d9/d61/tutorial_py_morphological_ops.html

❑ Opening (Apertura)

- ❑ Es una erosión seguida de una dilatación.

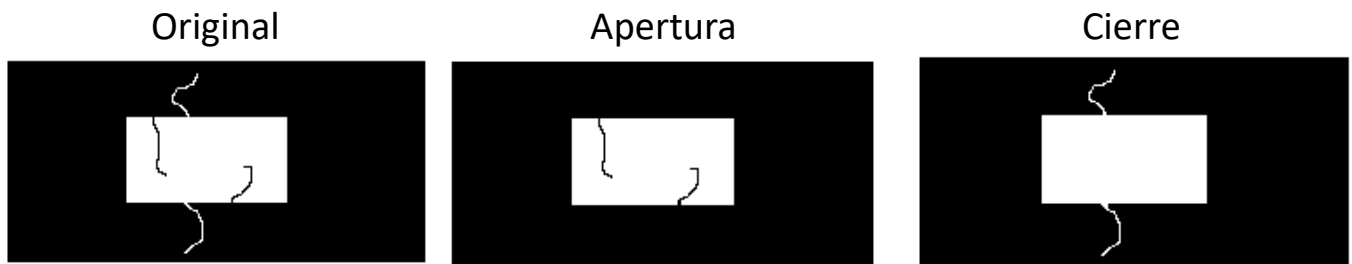
$$A \circ B = (A \ominus B) \oplus B$$

- ❑ **Elimina** pequeños **píxeles aislados** de la imagen, si el primer plano es brillante sobre un fondo oscuro.
- ❑ Satisface la propiedad de la **idempotencia**: después de la primera aplicación, otras aplicaciones no producen ningún cambio en el resultado.

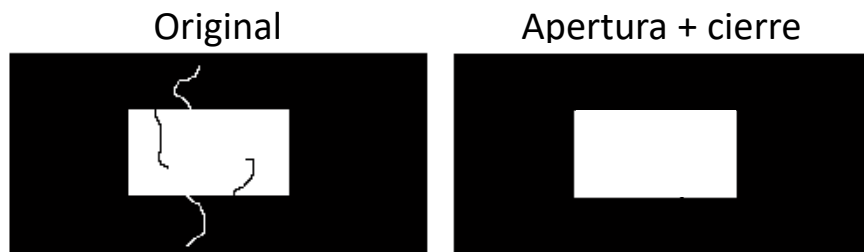


https://docs.opencv.org/3.2.0/d9/d61/tutorial_py_morphological_ops.html

Apertura y cierre



- Para deshacernos tanto de los objetos pequeños como de los agujeros pequeños tendremos que **ejecutar secuencialmente apertura y cierre**. El **orden importa**: apertura seguida de cierre y cierre seguido de apertura no producen, en general, el mismo resultado.

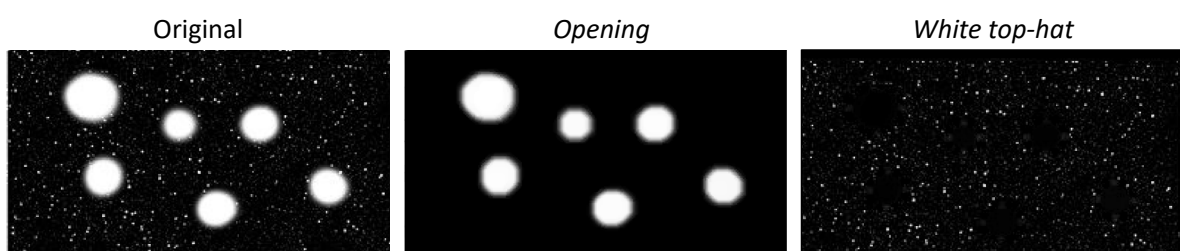


Top-hat

- La **apertura** y el **cierre** pueden **eliminar determinadas estructuras pequeñas**.
- Las transformaciones **top-hat** permiten **recuperar o resaltar** las estructuras eliminadas. Se utilizan para resaltar detalles blancos en fondos complejos (p. ej., en imágenes médicas), para extraer características (*feature extraction*), para mejorar imágenes (*image enhancement*), etc.
- Top-hat (white top-hat)**
 - Es la diferencia entre la imagen original (A) y una apertura ($A \circ B$).

$$I = A - A \circ B$$

- Permite extraer estructuras más brillantes que su entorno y pequeñas respecto al elemento estructurante:
 - Más pequeñas que el elemento estructurante, y
 - Más brillantes que sus alrededores.



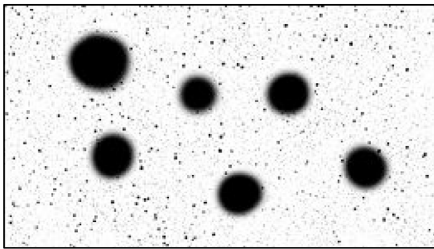
❑ *Bottom-hat (Black top-hat, black-hat)*

- ❑ Es la diferencia entre un cierre ($A \bullet B$) y la imagen original (A).

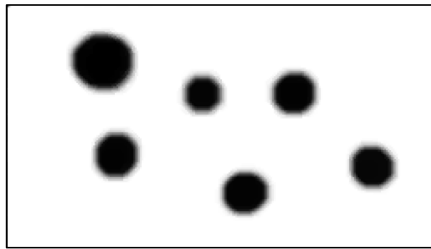
$$I = (A \bullet B) - A$$

- ❑ Permite enfatizar objetos de la imagen de entrada que sean pequeños y más oscuros que su entorno:
 - ❑ **Más pequeños** que el elemento estructurante, y
 - ❑ **Más oscuros** que sus alrededores.

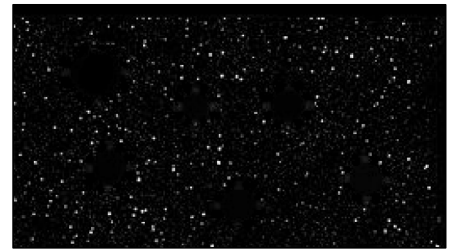
Original



Closing



Black top-hat



https://en.wikipedia.org/wiki/Top-hat_transform